

Техническое задание WebStyle на проект интеграции CRM "WebStyle" с сервисом MadeTask

Введение

Компания "WebStyle" специализируется на разработке сайтов "под ключ". У них есть команда штатных сотрудников (разработчики, менеджеры), работающих в CRM, и внешние подрядчики (дизайнеры, копирайтеры), с которыми коммуникация происходит через мессенджеры.

Существующие проблемы включают:

- **Разрозненность задач:** Задачи для внешних исполнителей ставятся в чатах, что приводит к потере информации и отсутствию единого трекера.
- **Ручные выплаты:** Бухгалтерия вручную осуществляет выплаты подрядчикам, сверяя реквизиты из Excel, что отнимает время и повышает риск ошибок.
- **Непрозрачность финансов:** Отсутствие оперативной информации о фактических выплатах по проектам до получения отчета из банка.

Цель проекта – интегрировать CRM "WebStyle" с сервисом MadeTask, чтобы создать единую систему управления задачами и автоматизировать процесс выплат подрядчикам. Это должно снизить нагрузку на бухгалтерию, повысить прозрачность и улучшить контроль за проектами. Подчеркивается, что взаимодействие с MadeTask будет происходить программно, через API, чтобы внешние подрядчики продолжали работать в привычном интерфейсе CRM.

1. Функциональные требования

1.1 Управление задачами (создание, обновление, статус)

1.2 Внешние исполнители должны иметь возможность отмечать задачу как "Готово" в CRM.

1.3 Менеджер должен иметь возможность подтверждать выполнение задачи в CRM.

1.4 Внешний исполнитель должен зарегистрироваться в системе для выполнения задач.

1.4.1 Автоматическое создание задач

○ Информация о задаче из CRM (например, описание, дата выполнения, приоритет, контактное лицо, связанные документы) должна автоматически передаваться в задачу MadeTask.

○ Необходима возможность задания дефолтных значений полей задачи для тех, у которых нет соответствующих полей в CRM.

○ Необходима возможность прикрепления файлов из CRM к задаче в MadeTask.

1.4.2 Синхронизация статуса задач

○ Изменения статуса задачи в MadeTask (например, "В работе", "Выполнено", "Отменено") должны автоматически синхронизироваться со статусом соответствующего в CRM.

○ Должна быть возможность настройки соответствия статусов MadeTask и статусов CRM.

○ Изменения статуса в CRM должны инициировать изменение статуса задачи в MadeTask.

1.4.3 Обновление информации о задаче

- Изменения информации о задаче в CRM (например, изменение даты выполнения, приоритета, контактного лица) должны автоматически обновлять информацию о соответствующей задаче в MadeTask.

- Изменения в описании задачи в MadeTask должны автоматически обновлять соответствующие поля в CRM.

- Должна обеспечиваться двусторонняя синхронизация комментариев к задаче в MadeTask и комментариев к задаче в CRM.

1.5 Управление пользователями (исполнители)

1.5.1 Сопоставление пользователей

- Должна быть возможность связать пользователей CRM с пользователями MadeTask (например, сопоставлять по email).

- При создании задачи из CRM, исполнителем в MadeTask должен назначаться соответствующий пользователь из списка сопоставленных.

- Если пользователь CRM не найден в MadeTask, система должна предоставить возможность создать нового пользователя в MadeTask.

1.5.2 Синхронизация ролей и разрешений

- Роли и права доступа пользователей CRM должны быть отражены в MadeTask (например, если пользователь является администратором в CRM, он должен обладать соответствующими правами в MadeTask).

1.5.3 Уведомление пользователей

- При назначении задачи в MadeTask пользователю, связанному с пользователем CRM, пользователь должен получать уведомление в CRM (настраивается).

- При изменении статуса задачи менеджеру должно отправляться уведомление.

- Бухгалтерия должна получать уведомление о назначенном платеже.

1.6 Управление платежами

1.6.1 Инициация платежей

- При изменении статуса задачи в MadeTask на "Выполнено" должна автоматически инициироваться выплата исполнителю и отправлена в бухгалтерию на согласование.

- Информация о сумме выплаты должна автоматически передаваться из MadeTask.

1.6.2 Синхронизация статуса платежей

- Статус платежа (например, "Создан", "Оплачен", "Отклонен") должен автоматически синхронизироваться между MadeTask и CRM.

- Должна быть возможность отслеживать историю платежей по каждой задаче в CRM.

1.6.3 Отчетность по платежам

- В CRM должна формироваться отчетность по всем выплатам, связанным с задачами в MadeTask.

1.7 Интерфейс и настройки

1.7.1 Интерфейс конфигурации

- Должен быть предоставлен интерфейс для настройки параметров интеграции, включая:

- Сопоставление полей CRM и MadeTask.
- Сопоставление статусов CRM и MadeTask.
- Настройки аутентификации и авторизации.
- Настройки уведомлений.
- Назначение ответственных за интеграцию.

- Просмотр логов интеграции.

1.7.2 Логирование

- Система должна вести подробный журнал (лог) всех операций интеграции, включая ошибки и предупреждения. Журнал должен быть доступен для администраторов.
- Необходима возможность фильтрации и поиска информации в логах.

1.7.3 Мониторинг

- В CRM должен отображаться статус интеграции с MadeTask (например, "Подключено", "Ошибка подключения").
- Система должна генерировать предупреждения при возникновении ошибок интеграции (например, невозможность создать задачу в MadeTask).

1.8 Безопасность и авторизация

1.8.1 Безопасность аутентификации

- Интеграция должна поддерживать безопасную аутентификацию через API MadeTask с использованием OAuth 2.0 или API ключей.

1.8.2 Управление доступом

- Доступ к функциям интеграции должен контролироваться на основе ролей пользователей в CRM.

1.9 Обработка ошибок

1.9.1 Обработка ошибок API

- Интеграция должна обрабатывать ошибки, возвращаемые API MadeTask и CRM, и отображать информативные сообщения об ошибках в CRM.

1.9.2 Механизм повторных попыток

- Интеграция должна иметь механизм повторных попыток для временных ошибок API (например, сетевые проблемы).

1.10 Отчетность

1.10.1 Отчет о создании задач

- Предоставление отчета о количестве автоматически созданных задач в MadeTask из CRM за определенный период времени.

1.10.2 Отчет о проблемах синхронизации

- Отчет о задачах, при синхронизации которых возникли ошибки и требуют ручного вмешательства.

1.10.3 Отчет о совершенных платежах

- Предоставлять в бухгалтерию отчет о совершенных платежах

2. Нефункциональные требования

2.1 Производительность

2.1.1 Время отклика:

- Среднее время отклика API (от CRM к MadeTask и обратно) не должно превышать 500 мс.
- Максимальное время отклика (95-й перцентиль) не должно превышать 1 секунду.

2.1.2 Пропускная способность:

Система должна выдерживать до 100 одновременных запросов (создание, обновление задач, получение информации о пользователе, создание платежа) без существенного ухудшения производительности (увеличение времени отклика более чем на 20%).

2.1.3 Масштабируемость:

Архитектура интеграции должна быть масштабируемой для поддержки увеличения количества пользователей, задач и транзакций. Масштабирование может быть обеспечено

путем горизонтального масштабирования серверов, использования распределенной базы данных или других соответствующих методов.

2.2 Надежность

2.2.1 Доступность:

Система должна быть доступна 24/7, с целевым уровнем доступности 99.9% в месяц (за исключением плановых технических работ).

2.2.2 Отказоустойчивость:

- Система должна быть устойчива к сбоям отдельных компонентов. Необходимо предусмотреть механизмы повторной отправки запросов при временной недоступности MadeTask.

- Для критически важных операций (например, создание задач, инициирование платежей) необходимо предусмотреть механизмы резервного копирования и восстановления данных.

2.2.3 Целостность данных:

Система должна гарантировать целостность данных, передаваемых между CRM и MadeTask. Необходимо использовать механизмы контроля целостности (например, контрольные суммы) защиты от потери данных.

2.2.4 Восстановление после сбоев:

В случае сбоя система должна автоматически восстанавливаться в течение 5 минут или менее. Автоматизация процесса восстановления должна быть обеспечена.

2.3. Безопасность

2.3.1 Аутентификация и авторизация:

- Доступ к API MadeTask должен быть защищен надежными механизмами аутентификации и авторизации (OAuth 2.0, API ключи).

- Необходимо обеспечить разделение прав доступа к API MadeTask для разных пользователей и ролей.

2.3.2 Защита данных при передаче:

Все данные, передаваемые между CRM и MadeTask, должны быть зашифрованы с использованием протокола HTTPS/TLS.

2.3.3 Защита хранимых данных:

Конфиденциальные данные (например, API ключи, пароли) должны быть надежно защищены от несанкционированного доступа (например, с использованием шифрования и контроля доступа).

2.3.4 Предотвращение атак:

Необходимо применять меры для защиты от распространенных веб-атак, таких как SQL-инъекции, межсайтовый скриптинг (XSS) и подделка межсайтовых запросов (CSRF).

2.3.5 Соответствие стандартам:

Система должна соответствовать требованиям безопасности PCI DSS, GDPR или другим применимым стандартам. Необходимо регулярно проводить аудиты безопасности и устранять выявленные уязвимости.

2.4 Удобство использования (Usability)

2.4.1 Простота настройки:

Интеграцию должно быть легко настроить и сконфигурировать. Необходимо предоставить руководства по настройке параметров интеграции.

2.4.2 Интуитивно понятный интерфейс:

Интерфейс интеграции в CRM должен быть интуитивно понятен. Важно обеспечить логичную организацию элементов интерфейса и понятные сообщения об ошибках.

2.4.3 Быстрый доступ к информации:

Пользователи должны иметь возможность быстро и легко найти необходимую информацию о статусе задач, выполненных платежах и других деталях, добавить фильтры для задач.

2.4.4 Обучаемость:

Необходимо предоставить обучающие материалы (например, видеоролики, инструкции) и обеспечить поддержку пользователей.

2.4.5 Адаптивность:

Интерфейс должен быть адаптивным к различным размерам экрана (desktop, mobile).

2.5 Сопровождение и поддержка

2.5.1 Логирование:

Система должна вести подробные логи всех операций и ошибок, необходимых для диагностики проблем и анализа производительности.

2.5.2 Мониторинг:

Необходимо настроить автоматический мониторинг системы для отслеживания доступности, производительности, ошибок и других важных параметров.

2.5.3 Техническая поддержка:

Должна быть предоставлена техническая поддержка для решения проблем и ответов на вопросы пользователей.

2.5.4 Документация:

Необходимо предоставить полную и актуальную техническую документацию, включая руководство пользователя, руководство по установке и настройке, описание API, схемы и диаграммы архитектуры системы, облегчающие понимания работы системы

2.6 Совместимость

2.6.1 Операционные системы:

Система должна быть совместима с операционными системами, используемыми в CRM и MadeTask (например, Windows, Linux, macOS).

2.6.2 Браузеры:

Интерфейс интеграции должен быть совместим с основными браузерами (например, Chrome, Firefox, Safari, Edge).

2.6.3 Версии API MadeTask:

Интеграция должна быть совместима с указанной версией API MadeTask. Необходимо обеспечить поддержку обратной совместимости или возможность обновления до новых версий API.

2.6.4 Базы данных:

Интеграция должна быть совместима с базами данных, используемыми в CRM (PostgreSQL).

2.7 Эксплуатационные требования

2.7.1 Установка и настройка:

Процесс установки и настройки должен быть описан в соответствующей инструкции.

2.7.2 Обновление:

Процесс обновления системы должен быть автоматизирован и не влиять на работоспособность системы.

2.7.3 Резервное копирование и восстановление:

Необходимо предусмотреть механизмы резервного копирования и восстановления данных для защиты от потери данных в случае сбоев.

2.7.4 Мониторинг:

Необходимо реализовать мониторинг ключевых показателей работы системы: загруженность сервера, время отклика, количество ошибок и др.

2.8 Нормативные требования

2.8.1 Законодательство:

Система должна соответствовать требованиям законодательства о защите персональных данных (например, GDPR), если она обрабатывает персональные данные пользователей.

2.8.2 Отраслевые стандарты:

Система должна соответствовать отраслевым стандартам безопасности и конфиденциальности, если таковые применимы к данной области.

3 Архитектура системы

В данном разделе рассматривается описание системы и процессов для разработки на основе анализа, а также детализация компонентов системы и их взаимодействие.

3.1 Описание бизнес-контекста

Описание диаграммы вариантов использования

- **Use Cases:**
 - **Создать задачу для подрядчика:** Менеджер создает задачу в CRM, задача автоматически создается в MadeTask.
 - **Назначить исполнителя на задачу:** Менеджер назначает внешнего исполнителя на задачу в CRM.
 - **Подтвердить выполнение задачи:** Менеджер подтверждает выполнение задачи в CRM. Система передает статус "Выполнено" в MadeTask.
 - **Автоматическая выплата:** MadeTask инициирует выплату внешнему исполнителю.
 - **Синхронизация статусов задач:** Обновление статусов задач между CRM и MadeTask.
 - **Регистрация исполнителя в MadeTask:** Отправка приглашения исполнителю для регистрации в MadeTask и отслеживание статуса регистрации (подтвержден/не подтвержден).
 - **Отчетность по выплатам:** Получение и отображение в CRM информации о произведенных выплатах из MadeTask.
- **Роли:**
 - **Менеджер проекта:** Создает задачи, назначает исполнителей, подтверждает выполнение задач.
 - **Разработчик CRM:** Реализует интеграцию с MadeTask API.
 - **Администратор CRM:** Управляет правами пользователей.
 - **Внешний исполнитель (Дизайнер/Копирайтер):** Выполняет задачи (работает в CRM с ограниченными правами).
 - **Бухгалтер:** контролирует итоговые данные о выплатах (данные хранятся в банке и бухгалтерской системе, но процесс подготовки выплат ускорен)

Визуализация описания диаграммы представлена в **приложении А**.

3.2 Анализ текущих процессов (As-Is)

3.2.1 Как сейчас работают с дизайнерами:

- **Как ставят задачи?** Менеджер проекта ставит задачу дизайнеру через мессенджер (например, Telegram, WhatsApp). Описание задачи, требования, сроки и примеры обычно передаются текстом и файлами (картинки, документы) в чате.
- **Как проверяют работу?** Дизайнер присылает результат работы (макет, эскиз) в чат. Менеджер проверяет, оставляет комментарии (также в чате). Может быть несколько итераций правок.
- **Как платят?** Дизайнер предоставляет реквизиты, которые бухгалтерия вносит в таблицу вручную. После завершения работы бухгалтерия берет данные из таблицы (часто из Excel-таблицы), создает платеж в банковском кабинете и отправляет деньги.

Условное изображение процесса выполнения задачи (As-Is) представлено на диаграмме в **приложении Б**.

3.2.2 Выявление болей:

- **Ручное управление задачами и возможная потеря информации:** Задачи, требования и комментарии теряются в истории чатов. Сложно отследить прогресс и историю изменений по задаче, особенно при большом числе итераций. Менеджеры тратят много времени на отслеживание задач, напоминания дизайнерам и пересылку файлов.
- **Отсутствие единого места хранения файлов:** Файлы с результатами работы разных подрядчиков разбросаны по разным чатам, что затрудняет поиск нужной версии.
- **Ручной ввод данных:** Бухгалтерия тратит много времени на ручной ввод реквизитов и сверку платежей. Высокий риск ошибок при вводе данных.
- **Задержки в оплате:** Процесс оплаты занимает время из-за ручной обработки, что может негативно сказываться на отношениях с подрядчиками.
- **Непрозрачность:** Сложно отследить, сколько фактически заплачено по конкретному проекту, пока не будет получен банковский отчет.
- **Сложность масштабирования:** Ручные процессы затрудняют масштабирование бизнеса и увеличение количества проектов.

3.3 Проектирование будущего решения (To-Be)

3.3.1 Роли пользователей определены выше в разделе 3.1.

3.3.2 Основные сценарии:

- **Менеджер создает задачу:** Менеджер создает задачу в CRM, указывая описание, сроки, прикладывая необходимые файлы, и назначает исполнителя. При назначении исполнителя (дизайнера/копирайтера) задача автоматически создается в MadeTask и направляется исполнителю.
- **Дизайнер выполняет задачу:** Дизайнер получает уведомление о новой задаче в CRM (в своем интерфейсе, с ограниченным набором прав). Выполняет задачу, загружает результаты в CRM и отмечает задачу как "Готово". После отметки о готовности результат направляется менеджеру.
- **Менеджер проверяет задачу:** Менеджер получает уведомление о выполненной задаче и проверяет результат работы дизайнера в CRM. Если все устраивает, подтверждает выполнение задачи. Если есть замечания, то возвращает задачу исполнителю на доработку.

- **Автоматическая выплата:** После подтверждения выполнения задачи, CRM отправляет запрос в MadeTask на выплату. MadeTask проводит выплату дизайнеру согласно привязанным реквизитам и создает отчет о выплате. Инициация выплаты автоматическая, но необходимо подтверждение бухгалтерии!

- **Отчетность:** CRM получает информацию о статусе платежа из MadeTask и отображает ее в системе. Отчет о выплате направляется в бухгалтерию.

3.3.3 Use Case Диаграмма приведена в **приложении В**.

3.3.4 Схема бизнес-процесса в BPMN приведена в **приложении Г**.

3.4 Интеграция систем

- **Сценарий 1: Создание задачи**

1. Менеджер создает задачу в CRM (указывает описание, сроки, исполнителя).
2. CRM получает данные задачи и исполнителя.
3. CRM отправляет запрос POST /tasks в MadeTask (см. API).
4. MadeTask создает задачу и возвращает ID созданной задачи в CRM.
5. CRM сохраняет ID задачи MadeTask в своей базе данных (для привязки).

- **Сценарий 2: Выполнение задачи и выплата**

1. Дизайнер отмечает задачу как "Готово" в CRM.
2. CRM отправляет запрос PUT /tasks/{taskId} в MadeTask (смена статуса).

3. MadeTask меняет статус задачи.

4. Менеджер подтверждает выполнение задачи в CRM.

5. CRM отправляет запрос POST /payments в MadeTask (см. API).

6. MadeTask иницирует выплату и возвращает ID платежа в CRM.

7. CRM сохраняет ID платежа и обновляет статус задачи.

- **Сценарий 3: Ошибка - у исполнителя нет счета в MadeTask**

1. Менеджер подтверждает выполнение задачи в CRM.
2. CRM отправляет запрос POST /payments в MadeTask.
3. MadeTask возвращает ошибку "No payment method found for user".
4. CRM получает ошибку и отображает уведомление менеджеру, что необходимо связаться с исполнителем для добавления платежных данных.
5. CRM устанавливает задаче статус "Ожидает добавления платежных данных".

3.5 Работа с API MadeTask

3.5.1 Методы API:

- **Создание задачи (POST /tasks):**

- **Параметры:**

- title (string, required): Заголовок задачи.

- description (string): Описание задачи.

- assignee_id (integer): ID исполнителя в MadeTask.

- price (float): Стоимость выполнения задачи.

- deadline (string): Срок выполнения задачи.

- metadata: дополнительные данные необходимые для идентификации задачи в CRM(опционально)

- **Пример запроса (JSON):**

- ```
json
```



```
{
 "title": "Разработка дизайна главной страницы",
 "description": "Создать дизайн главной страницы в соответствии с
брендбуком.",
 "assignee_id": "user_id_дизайнера",
 "deadline": "2025-09-21T17:00:00Z",
 "price": "1000.00",
 "metadata": {"crm_task_id" : "123"}
}
```

▪ **Пример ответа (JSON):**

```
json
{
 "id": 456, "
 title": "Разработать макет главной страницы",
 "status": "active"
}
```

○ **Изменение статуса задачи (PUT /tasks/{taskId}):**

▪ **Параметры:**

- task\_id (integer): ID задачи
- status (string, required): Новый статус задачи (active, completed, approved, rejected)

▪ **Пример запроса (JSON):** json { " task\_id ": "456", "status": "completed" }

○ **Отправка платежа (POST /payments):**

▪ **Параметры:**

- task\_id (integer): ID задачи.
- amount (float): Сумма выплаты.

▪ **Пример запроса (JSON):** json { "task\_id": 456, "amount": 100.00 }

▪ **Пример ответа (JSON):**

```
json
{
 "id": 789,
 "status": "pending", // Возможные статусы: pending, completed, failed
 "amount": 100.00
}
```

○ **Данные исполнителя:**

- Регистрацию и предоставление платежных реквизитов исполнитель проводит самостоятельно в MadeTask.
- Для проверки наличия платежных данных можно использовать отдельный запрос API. (Иначе, отлавливать ошибку при попытке создания платежа).

**Обязательные данные исполнителя:**

Для успешной работы с API MadeTask необходимо получить от исполнителя:

- id: Уникальный идентификатор пользователя в MadeTask. Можно получить после регистрации или через вызов API.
- payment\_info: Информация о платежных реквизитах. (необходимо уточнить у MadeTask, как они передаются и какие данные необходимы).

\* Позволяет проверить, зарегистрирован ли пользователь в сервисе MadeTask

▪ **Параметры (пример):** json { "id": "user\_id\_дизайнера" }

▪ **Ответ (пример):**

```
json
{
 "id": "user_id_дизайнера",
 "email": "email@test.ru",
}
```

```
"payment_info":
 {
 "account_number": "1234567890",
 "bank_name": "Tinkoff"
 }
}
```

### 3.5.2 Ограничения API:

- **Лимиты запросов:** Необходимо уточнить в документации MadeTask (или связаться с поддержкой) информацию о лимитах запросов в единицу времени (например, 100 запросов в минуту). Необходимо предусмотреть обработку ошибок, связанных с превышением лимитов (например, повторная отправка запроса через некоторое время).
- **Верификация:** Скорее всего, потребуется API-ключ или токен для авторизации запросов. Необходимо получить его и правильно настроить авторизацию в CRM.

## 3.6 Список задач для разработки

Рассмотреть для разработки следующие задачи:

1. **Реализация API-клиента для MadeTask:** Разработать модуль, который будет отвечать за взаимодействие с MadeTask API (авторизация, отправка запросов, обработка ответов).
2. **Добавление полей MadeTask ID в сущность "Задача":** Добавить поля для хранения ID задачи и ID платежа MadeTask в базе данных CRM.
3. **Реализация логики создания задачи в MadeTask:** При создании/назначении задачи, отправлять данные в MadeTask API.
4. **Реализация логики подтверждения выполнения задачи:** При подтверждении выполнения, отправлять запрос в MadeTask API.
5. **Реализация отображения статуса платежа:** Добавить UI-элементы для отображения статуса платежа в CRM.
6. **Обработка ошибок API:** Реализовать логику обработки ошибок, возвращаемых MadeTask API (например, отсутствие платежных данных, превышение лимитов).
7. **Создание роли для внешних подрядчиков:** Создание пользовательской роли с ограниченными правами для работы в CRM.

## 4 Технические условия

### 4.1. Требования к производительности

#### 4.1.2 Создание задачи:

- Время создания задачи в MadeTask через API CRM не должно превышать 2 секунды при нормальной нагрузке (до 10 запросов в секунду).
- Задержка между созданием задачи в CRM и появлением её в MadeTask не должна превышать 5 секунд.

#### 4.1.2. Обновление статуса задачи:

- Время обновления статуса задачи в MadeTask через API CRM не должно превышать 1 секунду.
- Задержка между изменением статуса задачи в CRM и изменением статуса в MadeTask (посредством вебхуков) не должна превышать 3 секунды в 95% случаев.

#### 4.1.3. Получение данных о пользователе:

- Время получения информации о пользователе из MadeTask через API CRM не должно превышать 0.5 секунды.

#### **4.1.4. Инициация платежа:**

- Время инициации платежа в MadeTask через API CRM не должно превышать 1 секунду.
- Уведомление о статусе платежа (полученное вебхуком) должно быть обработано в CRM в течение 2 секунд после отправки вебхука MadeTask.

#### **4.1.5. Обработка вебхуков:**

- Обработка каждого вебхука не должна занимать более 500мс.

### **4.2. Требования к надежности**

#### **4.2.1. Доступность:**

- Интеграция должна обеспечивать доступность 99.9% в месяц (за исключением плановых технических работ, о которых необходимо предупреждать заранее).
- Система должна автоматически восстанавливаться после сбоев в течение не более 5 минут.

#### **4.2.2. Обработка ошибок:**

- Все ошибки, возникающие при взаимодействии с API MadeTask, должны быть корректно обработаны и зарегистрированы в логах CRM.
- В случае недоступности MadeTask, CRM должна повторно отправлять запросы (с экспоненциальной задержкой) в течение определенного периода времени (например, 1 часа) или сохранять задачи в очередь для последующей отправки.

#### **4.2.3. Целостность данных:**

- Данные, передаваемые между CRM и MadeTask, должны быть гарантированно доставлены и не должны быть повреждены.
- Необходимо предусмотреть механизмы контроля целостности данных (например, контрольные суммы).

#### **4.2.4. Обработка дубликатов вебхуков:**

- CRM должна уметь определять и игнорировать дублирующиеся вебхуки.

### **4.3 Требования к безопасности**

#### **4.3.1. Аутентификация и авторизация:**

- Все API запросы к MadeTask должны быть аутентифицированы с использованием безопасного механизма (например, OAuth 2.0, API ключи).
- Необходимо обеспечить разделение прав доступа к API MadeTask (например, различные API ключи для разных операций).

#### **4.3.2. Защита передаваемых данных:**

- Все данные, передаваемые между CRM и MadeTask, должны быть зашифрованы с использованием протокола HTTPS/TLS.
- Необходимо избегать хранения конфиденциальной информации (например, паролей, API ключей) в открытом виде.

#### **4.3.3. Безопасность хранения данных:**

- Данные, хранящиеся в CRM, связанные с интеграцией MadeTask (например, ID задач MadeTask, ID пользователей MadeTask), должны быть защищены от несанкционированного доступа.

- Необходимо соблюдать требования законодательства о защите персональных данных.

#### **4.3.4. Валидация данных:**

- Перед отправкой данных в MadeTask необходимо проводить валидацию на стороне CRM, чтобы избежать инъекций и некорректных данных.

### **4.4. Требования к совместимости программного обеспечения**

#### **4.4.1. Версии API MadeTask:**

- Интеграция должна быть совместима с текущей версией API MadeTask (указана в документации MadeTask).

- Необходимо обеспечить возможность быстрой адаптации к новым версиям API MadeTask.

#### **4.4.2. Форматы данных:**

- Интеграция должна поддерживать форматы данных, используемые в API MadeTask (например, JSON).

- Необходимо обеспечить корректное преобразование данных между форматами CRM и MadeTask.

#### **4.4.3. Совместимость с системой CRM:**

- Интеграция должна быть совместима с версией CRM, указанной в проектной документации.

- Необходимо обеспечить отсутствие конфликтов с другими модулями и расширениями CRM.

#### **4.4.4. Поддержка языков программирования:**

- Интеграция должна быть реализована с использованием языка программирования совместимых с CRM.

#### **4.4.5 Независимость от платформы:**

- Интеграция должна быть выполнена таким образом, чтобы обеспечивалась независимость от используемой платформы.

## **5 Тестирование и приемка**

### **5.1 Тестирование**

**5.1.1 Модульное тестирование:** Каждый модуль интеграции должен быть протестирован отдельно для проверки его функциональности и корректности работы.

**5.1.2 Интеграционное тестирование:** Необходимо провести интеграционное тестирование для проверки взаимодействия между CRM и MadeTask.

**5.1.3 Нагрузочное тестирование:** Необходимо провести нагрузочное тестирование для оценки производительности и стабильности интеграции при различных уровнях нагрузки.

**5.1.4 Тестирование безопасности:**

- Необходимо провести тестирование безопасности для выявления уязвимостей и обеспечения защиты данных.
- Регулярно проводить сканирование безопасности кода

**5.1.5 Регрессионное тестирование:** При внесении изменений в существующий код и после любого обновления, необходимо проводить регрессионное тестирование для уверенности, что существующая функциональность не повреждена.

**5.2 Поддержка и сопровождение**

**Гарантийный срок:** Устанавливается гарантийный срок в соответствии с договором на разработку.

**Техническая поддержка:** Обеспечивается техническая поддержка в течение гарантийного срока.

**Обновления:** Выпуск обновлений для исправления ошибок и улучшения функциональности.

**5.3 Приемка**

**5.3.1. Критерии приемки.**

Интеграция должна соответствовать всем требованиям, указанным в данном техническом задании. Допускается корректировка требований по согласованию с Заказчиком.

Не должно быть критических ошибок, влияющих на работоспособность интеграции.

При приемке необходимо предоставить:

- подробную техническую документацию по интеграции, включая описание архитектуры, используемых технологий, API, схемы взаимодействия, алгоритмы обработки ошибок и инструкции по установке/настройке.
- руководство пользователя, описывающее процесс работы с интеграцией для конечных пользователей CRM.
- детальное описание API взаимодействия между CRM и MadeTask, включая примеры запросов и ответов.

**5.3.2 Процедура приемки.**

- Заказчик проводит тестирование интеграции в соответствии с планом тестирования.
- При обнаружении ошибок, разработчик устраняет их в установленные сроки.
- После устранения ошибок, проводится повторное тестирование.
- При успешном завершении тестирования, подписывается акт приемки-сдачи.

## 6 График выполнения проекта

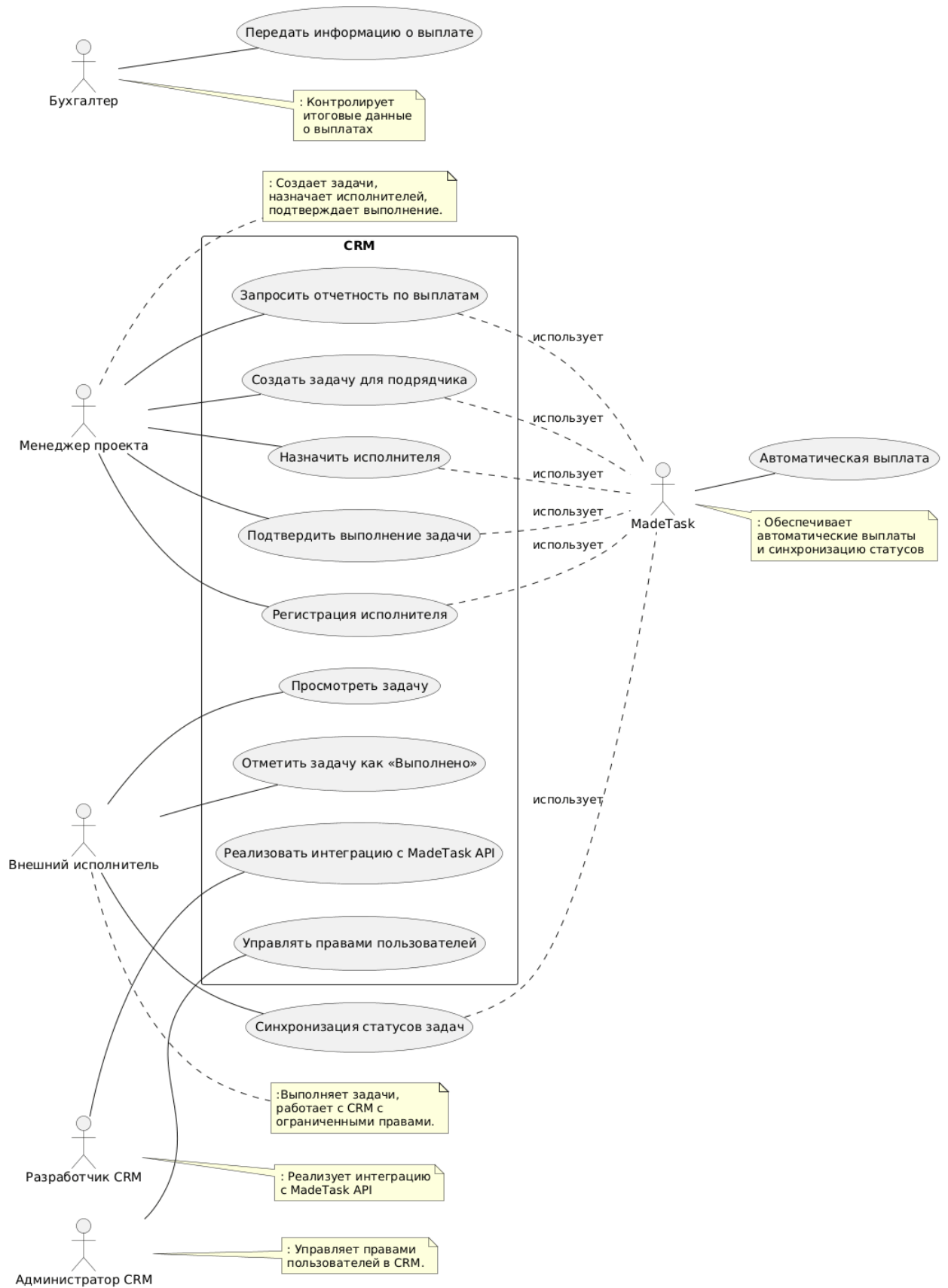
| Этап                                                                            | Начало     | Окончание  | Длительность (нед.) | Ответственный                         | Результат                                                                    |
|---------------------------------------------------------------------------------|------------|------------|---------------------|---------------------------------------|------------------------------------------------------------------------------|
| <b>1. Анализ и Планирование (Недели 1-2)</b>                                    |            |            |                     |                                       |                                                                              |
| 1.1 Сбор и анализ требований от заинтересованных сторон                         | 22.09.2025 | 26.09.2025 | 1                   | Аналитик                              | Документ с согласованными требованиями (функциональными и нефункциональными) |
| 1.2 Оценка текущей инфраструктуры CRM и MadeTask                                | 22.09.2025 | 26.09.2025 | 1                   | Архитектор, DevOps                    | Отчет об оценке инфраструктуры                                               |
| 1.3 Разработка технического задания (ТЗ) и спецификаций                         | 29.09.2025 | 03.10.2025 | 1                   | Аналитик, Архитектор                  | Утвержденное ТЗ                                                              |
| 1.4 Планирование проекта (ресурсы, сроки, риски)                                | 29.09.2025 | 03.10.2025 | 1                   | Project Manager                       | Утвержденный план проекта, график и матрица рисков                           |
| <b>2. Проектирование и Разработка (Недели 3-7)</b>                              |            |            |                     |                                       |                                                                              |
| 2.1 Проектирование архитектуры интеграции                                       | 06.10.2025 | 10.10.2025 | 1                   | Архитектор                            | Схема архитектуры интеграции                                                 |
| 2.2 Разработка API для интеграции между CRM и MadeTask                          | 06.10.2025 | 24.10.2025 | 3                   | Разработчики                          | Программный код API                                                          |
| 2.3 Разработка логики интеграции (создание задач, обновление статусов, выплаты) | 13.10.2025 | 31.10.2025 | 3                   | Разработчики                          | Программный код логики интеграции                                            |
| 2.4 Разработка пользовательского интерфейса (UI) в CRM (если требуется)         | 20.10.2025 | 31.10.2025 | 2                   | UI/UX Дизайнер, Front-end Разработчик | Макеты и компоненты пользовательского интерфейса                             |
| 2.5 Написание модульных тестов                                                  | 27.10.2025 | 07.11.2025 | 2                   | Разработчики, Тестировщики            | Набор модульных тестов                                                       |
| <b>3. Тестирование (Недели 8-9)</b>                                             |            |            |                     |                                       |                                                                              |
| 3.1 Проведение модульного тестирования                                          | 03.11.2025 | 07.11.2025 | 1                   | Разработчики, Тестировщики            | Результаты модульного тестирования                                           |

| Этап                                                                           | Начало     | Окончание  | Длительность (нед.) | Ответственный                      | Результат                                        |
|--------------------------------------------------------------------------------|------------|------------|---------------------|------------------------------------|--------------------------------------------------|
| 3.2 Проведение интеграционного тестирования                                    | 10.11.2025 | 14.11.2025 | 1                   | Тестировщики                       | Результаты интеграционного тестирования          |
| 3.3 Проведение нагрузочного тестирования                                       | 17.11.2025 | 21.11.2025 | 1                   | Тестировщики, DevOps               | Результаты нагрузочного тестирования             |
| 3.4 Тестирование безопасности                                                  | 17.11.2025 | 21.11.2025 | 1                   | Тестировщики безопасности          | Отчет о тестировании безопасности                |
| 3.5 Исправление обнаруженных дефектов                                          | 03.11.2025 | 21.11.2025 | 3                   | Разработчики                       | Исправленные дефекты                             |
| <b>4. Внедрение и Развертывание (Недели 10-11)</b>                             |            |            |                     |                                    |                                                  |
| 4.1 Подготовка инфраструктуры (серверы, базы данных)                           | 24.11.2025 | 28.11.2025 | 1                   | DevOps                             | Настроенная инфраструктура                       |
| 4.2 Развертывание интеграции на тестовом окружении                             | 24.11.2025 | 28.11.2025 | 1                   | DevOps, Разработчики               | Развернутая интеграция на тестовом окружении     |
| 4.3 Проведение приемочного тестирования (UAT)                                  | 01.11.2025 | 05.11.2025 | 1                   | Заказчик, Тестировщики             | Пройдено приемочное тестирование                 |
| 4.4 Развертывание интеграции на продуктивном окружении                         | 08.12.2025 | 12.12.2025 | 1                   | DevOps, Разработчики               | Развернутая интеграция на продуктивном окружении |
| <b>5. Завершение и Документация (Неделя 12)</b>                                |            |            |                     |                                    |                                                  |
| 5.1 Подготовка и передача документации (техническая, руководство пользователя) | 01.12.2025 | 12.12.2025 | 2                   | Технический писатель, Разработчики | Комплект документации                            |
| 5.2 Обучение пользователей (если необходимо)                                   | 08.12.2025 | 12.12.2025 | 1                   | Ответственный за обучение          | Проведено обучение                               |
| 5.3 Закрытие проекта и подготовка отчета                                       | 15.12.2025 | 19.12.2025 | 1                   | Project Manager                    | Закрытый проект, отчет о завершении              |

*\* Допускается корректировка графика и сроков выполнения по согласованию с Заказчиком.*

# Приложение А

## Диаграмма вариантов использования

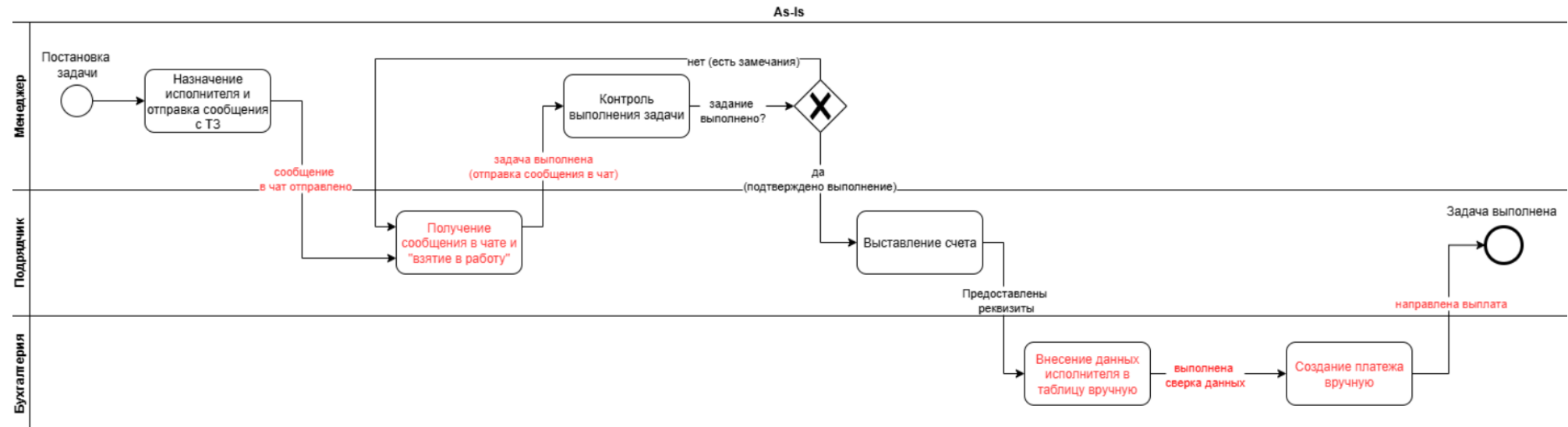




## Приложение Б

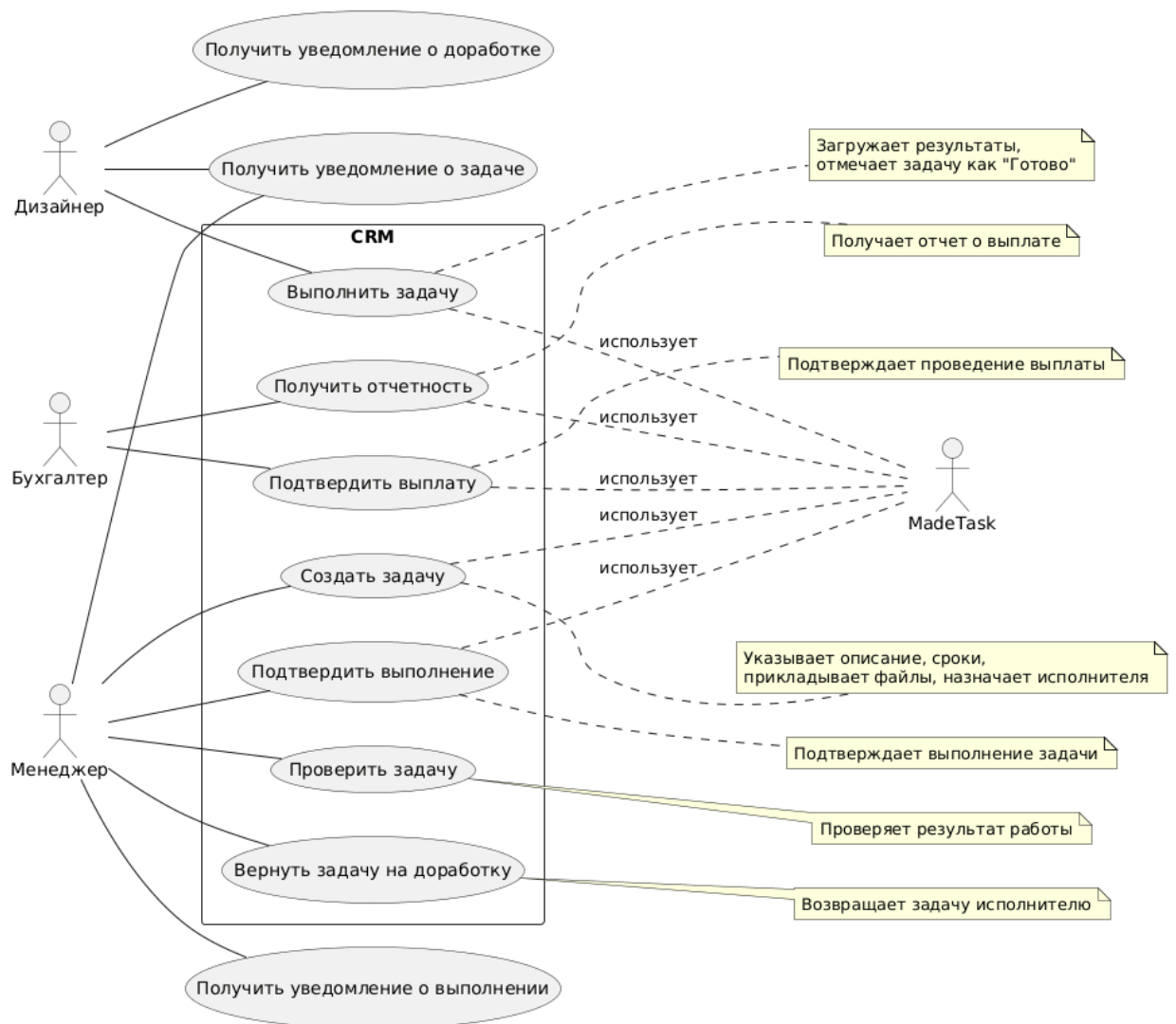
### Диаграмма выполнения задач As-Is

Красным цветом выделены места возможных «болей».



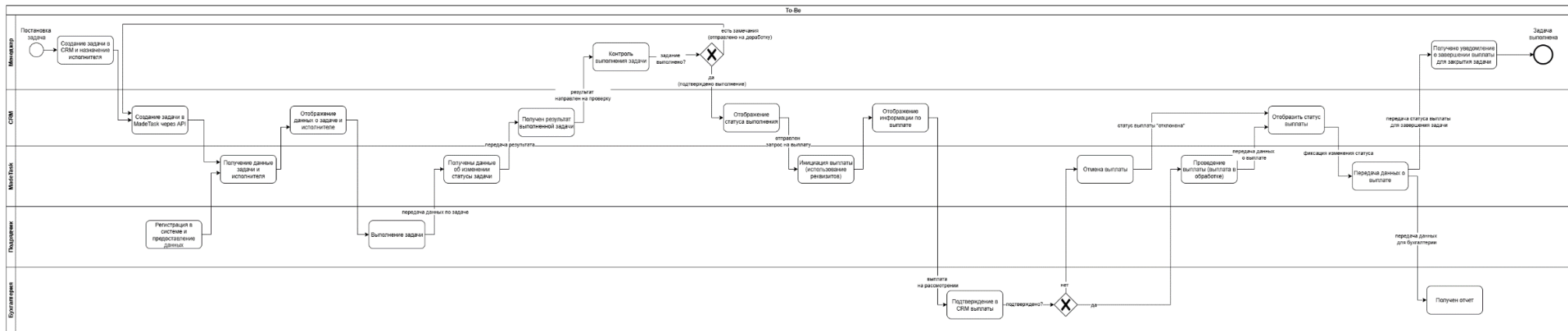
## Приложение В

### Use Case диаграмма



## Приложение Г

### Схема BPMN бизнес-процесса



## Приложение Д

### Диаграмма последовательностей (UML)

